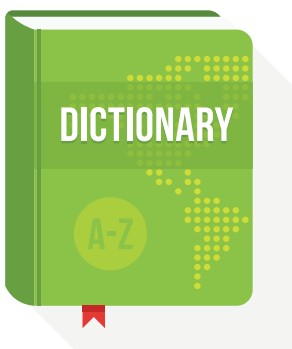
```
>>> set1 > set2 #Test for a proper superset
True
>>> set1 > set1 #A set is never a proper subset of itself
False
```

Set Usage

While solving real-life problems, sets come in handy when we want to eliminate all duplicates from our data structure. Let us consider an example of a college, where a student might attend or bunk multiple lectures in a day, with the condition that if a student comes to the college he/she will attend at least one lecture. Now that we have the attendance data of the students and we wish to find out how many students actually came to the college, we can simply build a set of students from the attendance of all lectures. The set will eliminate all possible duplicates since the students who attended more than one lecture will appear in multiple attendance lists.

```
>>> english = ['Rahul', 'Tia', 'Jia','Raj', 'Dave'] #attendance list
>>> maths = ['Rahul', 'Ram', 'Tia', 'Sia', 'Raj']   #attendance list
>>> visited = set(maths+english) #set of students who visited college
>>> print(visited)
{'Tia', 'Dave', 'Jia', 'Sia', 'Raj', 'Rahul', 'Ram'} #no duplicates
```

Dictionary



Dictionary also know as dict, is the python implementation of associative arrays. These are unordered `[key, value]` pairs similar to a `HashMap` in Java, wherein a key cannot appear more than once in the collection and key should belong to an immutable type. This key is mapped to a value which can be of any Python data type.

A dictionary can be initialized by giving a set of key-value pairs enclosed within curly brackets or using a dict constructor which can either be called with no arguments or by passing a sequence of key-value pairs.